

Lab 10. OOP and Collections Framework Exercises

Writing Good Programs

The only way to learn programming is program, program and program. Learning programming is like learning cycling, swimming or any other sports. You can't learn by watching or reading books. Start to program immediately. On the other hands, to improve your programming, you need to read many books and study how the masters program.

It is easy to write programs that work. It is much harder to write programs that not only work but also easy to maintain and understood by others – I call these good programs. In the real world, writing program is not meaningful. You have to write good programs, so that others can understand and maintain your programs.

Pay particular attention to:

1. Coding style:

- Read Java code convention: "Google Java Style Guide" or "Java Code Conventions - Oracle".
- Follow the Java Naming Conventions for variables, methods, and classes STRICTLY. Use CamelCase for names. Variable and method names begin with lowercase, while class names begin with uppercase. Use nouns for variables (e.g., radius) and class names (e.g., Circle). Use verbs for methods (e.g., getArea(), isEmpty()).
- **Use Meaningful Names:** Do not use names like a, b, c, d, x, x1, x2, and x1688 - they are meaningless. Avoid single-alphabet names like i, j, k. They are easy to type, but usually meaningless. Use single-alphabet names only when their meaning is clear, e.g., x, y, z for co-ordinates and i for array index. Use meaningful names like row and col (instead of x and y, i and j, x1 and x2), numStudents (not n), maxGrade, size (not n), and upperbound (not n again). Differentiate between singular and plural nouns (e.g., use books for an array of books, and book for each item).
- Use consistent indentation and coding style. Many IDEs (such as Eclipse / NetBeans) can re-format your source codes with a single click.

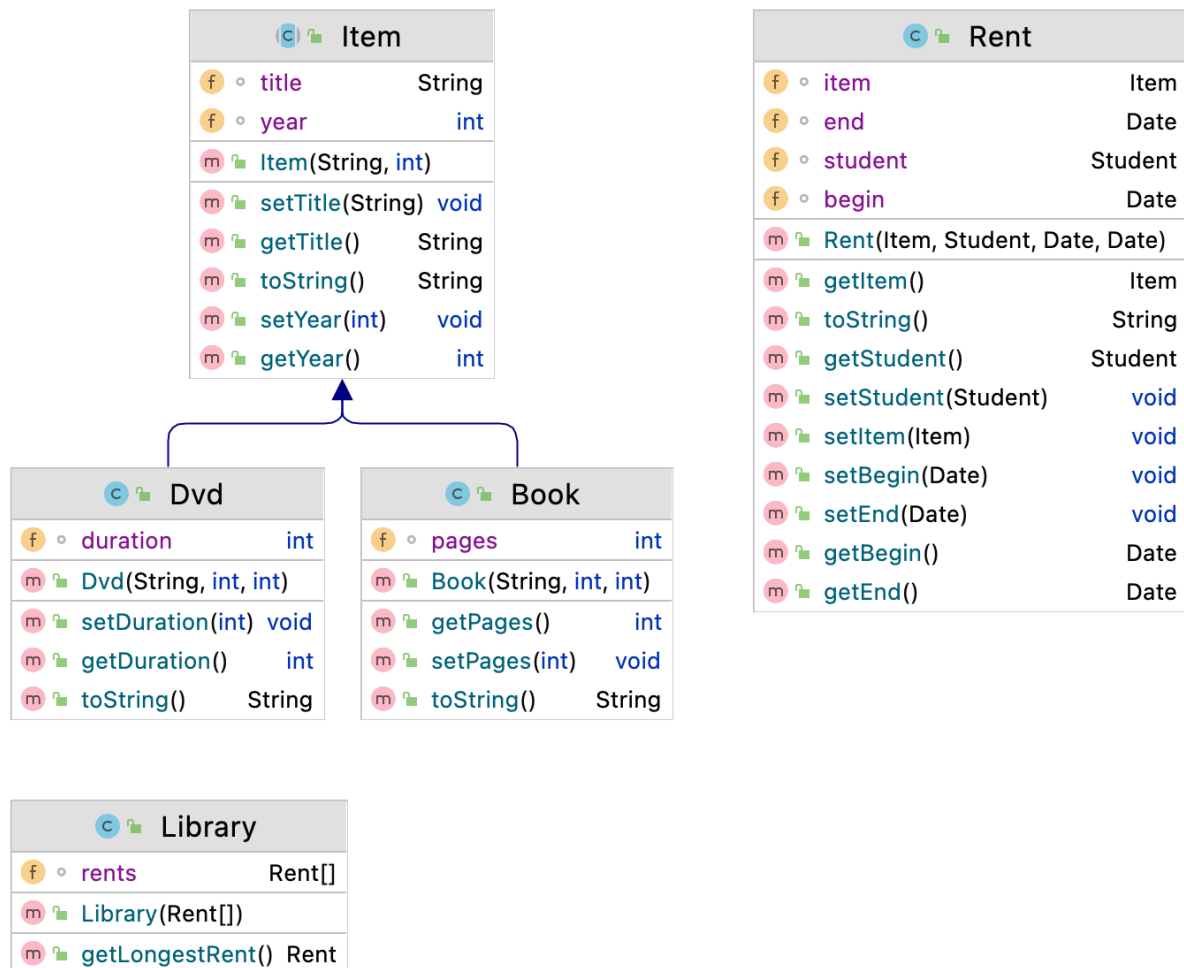
2. Program Documentation: Comment! Comment! and more Comment to explain your code to other people and to yourself three days later.

3. The only way to learn programming is program, program and program on challenging problems. The problems in this tutorial are certainly NOT challenging. There are tens of thousands of challenging problems available – used in training for various programming contests (such as International Collegiate Programming Contest (ICPC), International Olympiad in Informatics (IOI)).

1 Exercises on OOP

1.1 Library

Write code for an application designed as shown in the following class diagram.



```

1 package com.oop.library ;
2
3 /**
4  * Class generalizing Books and DVDs
5  */
6 public abstract class Item {
7     String title ;
8     int year ;
9
10    public Item(String title , int year) {

```



```
11  /* TODO */  
    }  
13  
    /* TODO */  
15  
    @Override  
17    public String toString() {  
        return "Item[" + "title=" + title + '\'  
19        + ", year=" + year + ']';  
    }  
21 }
```



```
1  package com.oop.library;  
  
3  /**  
    * Class representing a DVD  
5  */  
    public class Dvd extends Item {  
7        int duration;  
  
9        public Dvd(String title, int year, int duration) {  
            /* TODO */  
11        }  
  
13        /* TODO */  
  
15        @Override  
        public String toString() {  
17            return "Dvd[" + "duration=" + duration  
                + ", title=" + title + '\'  
19            + ", year=" + year + ']';  
        }  
21 }
```



```
1  package com.oop.library;  
  
3  /**  
    * Class representing a Book  
5  */  
    public class Book extends Item {  
7        int pages;  
  
9        public Book(String title, int year, int pages) {
```



```

11     }
    /* TODO */

13     /* TODO */

15     @Override
    public String toString() {
17         return "Book[" + "pages=" + pages
            + ", title=" + title + '\ '
19         + ", year=" + year + ']' ;
    }
21 }

```



```

1 package com.oop.library;

3 import java.util.Date;

5 /**
   * Rent implements a rent of an Item for a delimited time frame
7  */
   public class Rent {
9       Item item;
       Student student;
11      Date begin;
       Date end;

13
       public Rent(Item item, Student student, Date begin, Date end) {
15         /* TODO */
       }

17         /* TODO */

19
       @Override
21      public String toString() {
           return "Rent[" + "item=" + item
23             + ", student=" + student
           + ", begin=" + begin
25             + ", end=" + end + ']' ;
       }
27 }

```



```

1 package com.oop.library;

```



```
3 import java.util.Objects;

5 public class Student implements Comparable<Student> {
    String name;
    String lastname;
    String phone;
    double average;

11     public Student(String name, String lastname, String phone) {
        /* TODO */
13     }

15     public Student(String name, String lastname, double average) {
        /* TODO */
17     }

19     public Student(String name, String lastname, String phone, double
        ↪ average) {
        /* TODO */
21     }

23     /* TODO */

25     @Override
    public int compareTo(Student s) {
27         /* TODO */
    }

29

31     @Override
    public boolean equals(Object o) {
        if (this == o) {
33             return true;
        }

35         if (o == null || getClass() != o.getClass()) {
37             return false;
        }

39         Student student = (Student) o;
41         return Double.compare(student.average, average) == 0
            && Objects.equals(name, student.name)
43             && Objects.equals(lastname, student.lastname)
            && Objects.equals(phone, student.phone);
45     }

47     @Override
    public int hashCode() {
49         return Objects.hash(name, lastname, phone, average);
    }

51

53     @Override
    public String toString() {
```



```

return "Student[" +
55     "name=" + name + '\ ' +
        ", lastname=" + lastname + '\ ' +
57     ", phone=" + phone + '\ ' +
        ", average=" + average +
59     "]" ;
    }
61 }

```



```

1 package com.oop.library ;

3 import java.text.ParseException ;
  import java.text.SimpleDateFormat ;
5 import java.util.Locale ;

7 /**
   * Implement the classes described in UML diagram.
9   * Test them with the following main.
   */
11 public class TestApp {
    public static void main(String[] args) throws ParseException {
13         Item i1 = new Book("Soffocare", 2002, 170);
        Item i2 = new Dvd("Moon", 2011, 130);

15
        Student s1 = new Student("0001", "Darrell", "Abbott");
17         Student s2 = new Student("0002", "Nick", "Drake");

19         SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy", Locale.
            ↪ ENGLISH);
        Rent[] rents = new Rent[5];
21         rents[0] = new Rent(i1, s1, sdf.parse("15/06/2020"), sdf.parse("
            ↪ 15/07/2020"));
        rents[1] = new Rent(i1, s2, sdf.parse("10/07/2020"), sdf.parse("
            ↪ 20/07/2020"));
23         rents[2] = new Rent(i1, s1, sdf.parse("25/08/2020"), sdf.parse("
            ↪ 14/11/2020"));
        rents[3] = new Rent(i2, s2, sdf.parse("10/07/2020"), sdf.parse("
            ↪ 20/07/2020"));
25         rents[4] = new Rent(i2, s1, sdf.parse("25/08/2020"), sdf.parse("
            ↪ 28/08/2020"));

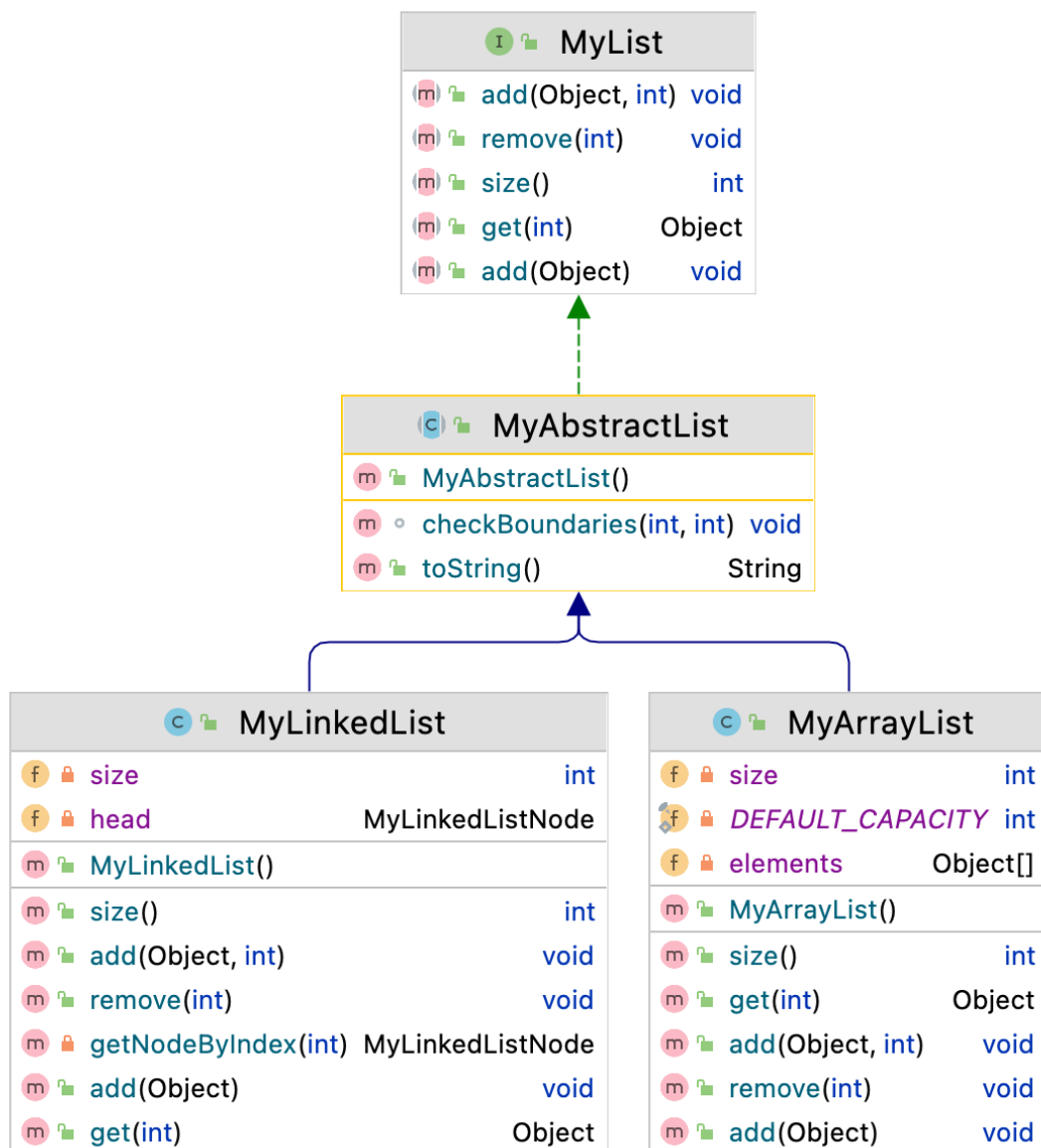
27         Library l = new Library(rents);
        System.out.println(l.getLongestRent());
29     }
}

```

2 Exercises on Collections

2.1 MyList

Write code for an application designed as shown in the following class diagram.



```

package com.oop.collections.mylis;

2 public interface MyList {
4     void add(Object o);
  
```



```

void add(Object o, int index);
6 void remove(int index);
  Object get(int index);
8  int size();
  }

```



```

1 package com.oop.collections.mylist;

3 public abstract class MyAbstractList implements MyList {
    void checkBoundaries(int index, int limit) {
5        if (index < 0 || index > limit) {
            throw new ArrayIndexOutOfBoundsException();
7        }
    }
9
    @Override
11   public String toString() {
        StringBuilder sb = new StringBuilder();
13        for (int i = 0; i < size(); i++) {
            sb.append(String.format("[%s]", get(i).toString()));
15        }
        return sb.toString();
17   }
}

```



```

package com.oop.collections.mylist;

2
/**
4  * Implementation of a simplified ArrayList
  */
6 public class MyArrayList extends MyAbstractList {
    static final int INITIAL_SIZE = 16;
8    Object[] elements;
    int size;

10
    public MyArrayList() {
12        elements = new Object[INITIAL_SIZE];
        size = 0;
14    }

16    @Override
    public void add(Object o) {
18        if (size >= elements.length - 1) {

```




```

    enlarge();
20     }
    elements[size++] = o;
22 }

24 @Override
    public void add(Object o, int index) {
26     /* TODO */
    }

28 @Override
30     public Object get(int index) {
        checkBoundaries(index, size - 1);
32     return elements[index];
    }

34 @Override
36     public void remove(int index) {
        /* TODO */
38     }

40 @Override
    public int size() {
42     /* TODO */
    }

44 void enlarge() {
46     Object[] tmp = new Object[elements.length * 2];
        System.arraycopy(elements, 0, tmp, 0, elements.length);
48     elements = tmp;
    }
50 }

```



```

package com.oop.collections.mylist;

2 /**
4  * Implementation of a single node composing the linked list
   */
6 public class MyLinkedListNode {
    Object payload;
    MyLinkedListNode next;

10     public MyLinkedListNode(Object payload) {
        /* TODO */
12     }

14     public MyLinkedListNode(Object payload, MyLinkedListNode next) {

```



```
16 }  
    /* TODO */  
18 public Object getPayload() {  
    /* TODO */  
20 }  
22 public void setPayload(Object dataValue) {  
    /* TODO */  
24 }  
26 public MyLinkedListNode getNext() {  
    /* TODO */  
28 }  
30 public void setNext(MyLinkedListNode nextValue) {  
    /* TODO */  
32 }  
}
```



```
1 package com.oop.collections.mylist;  
  
3 /**  
 * Implementation of a simplified LinkedList class  
5 */  
public class MyLinkedList extends MyAbstractList {  
7     MyLinkedListNode head;  
    int size;  
9  
    public MyLinkedList() {  
11         head = null;  
        size = 0;  
13    }  
  
15    private MyLinkedListNode getNodeByIndex(int index) {  
        MyLinkedListNode current = head;  
17        for (int i = 0; i < index; i++) {  
            current = current.getNext();  
19        }  
        return current;  
21    }  
  
23    @Override  
    public void add(Object o) {  
25        add(o, size);  
    }  
27 }
```



```

@Override
29 public void add(Object o, int index) {
    checkBoundaries(index, size);
31     if (index == 0) {
        head = new MyLinkedListNode(o, head);
33     } else {
        MyLinkedListNode current = getNodeByIndex(index - 1);
35         current.setNext(new MyLinkedListNode(o, current.getNext()));
    }
37     size++;
    }
39
    @Override
41 public Object get(int index) {
    /* TODO */
43 }

45 @Override
    public void remove(int index) {
47     /* TODO */
    }
49

    @Override
51 public int size() {
    /* TODO */
53 }
}

```



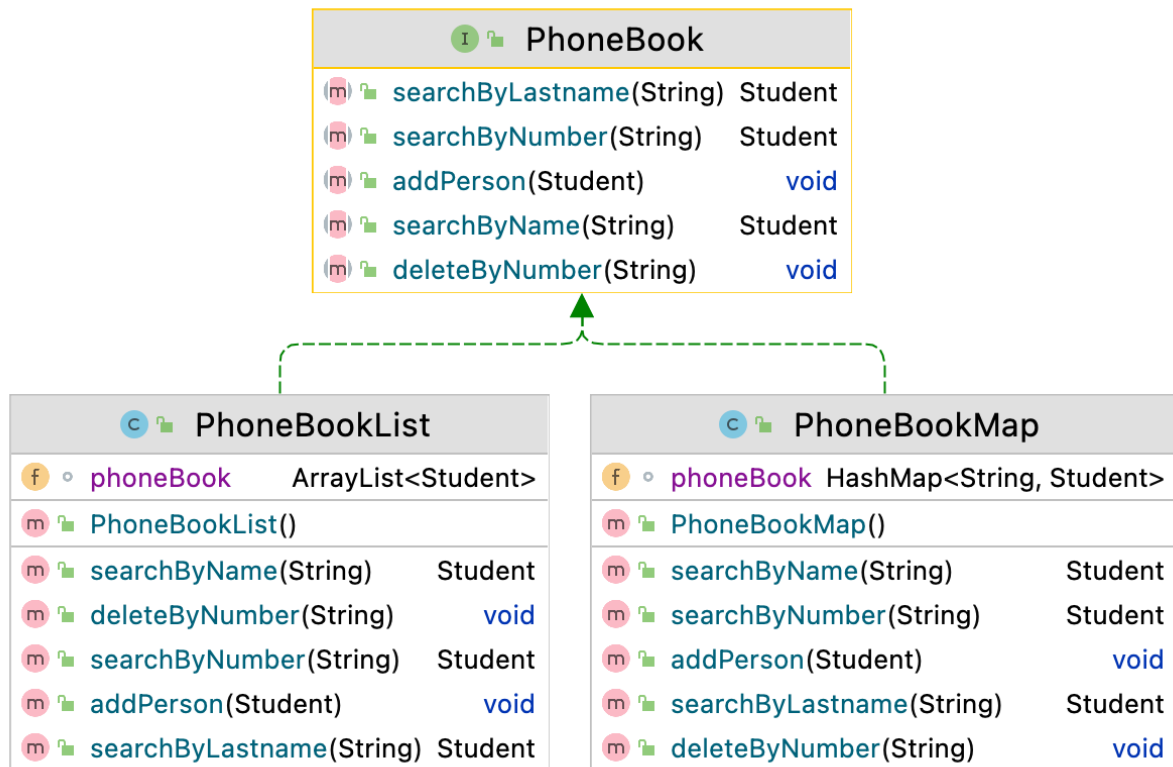
```

package com.oop.collections.mylis;
2
/**
4  * Provide an implementation of the MyList interface
  * test it with the main() function reported below
6  */
public class TestApp {
8     public static void main(String[] args) {
        MyList l = new MyArrayList();
10         l.add("a", 0);
        l.add("b");
12         l.add("c", 0);
        l.add("c", 3);
14         l.remove(3);
        System.out.println(l.size());
16         System.out.println(l);
    }
18 }

```

2.2 PhoneBook

Write code for an application designed as shown in the following class diagram.



```

package com.oop.collections.phonebook;

import java.util.Objects;

public class Student implements Comparable<Student> {
    /* TODO */

    public Student(String name, String lastname, String phone) {
        /* TODO */
    }

    public Student(String name, String lastname, double average) {
        /* TODO */
    }

    public Student(String name, String lastname, String phone, double
        ↪ average) {
        /* TODO */
    }
}
  
```



```

20  /* TODO */

22  @Override
    public int compareTo(Student s) {
24      /* TODO */
    }

26

28  @Override
    public int hashCode() {
        return Objects.hash(name, lastname, phone, average);
30  }

32  @Override
    public boolean equals(Object o) {
34      /* TODO */
    }

36

38  @Override
    public String toString() {
        return "Student[" +
40            "name='" + name + '\', ' +
            ", lastname='" + lastname + '\', ' +
42            ", phone='" + phone + '\', ' +
            ", average='" + average +
44            ']' ;
    }
46 }

```



```

package com.oop.collections.phonebook;

2
/**
4  * Interface representing a generic PhoneBook Implementing classes
  * must provide methods for inserting, deleting and searching persons
6  * within the PhoneBook
  */
8  public interface PhoneBook {
    /**
10     * Add a person to the PhoneBook
    *
12     * @param p The person to be added to the PhoneBook
    */
14     void addPerson(Student p);

16     /**
    * Search a person within the PhoneBook by name
18     *

```



```
20  * @param name The name to be searched
    * @return The person found, null otherwise
    */
22  Student searchByName(String name);

24  /**
    * Search a person within the PhoneBook by lastname
26  *
    * @param lastname The lastname to be searched
28  * @return The person found, null otherwise
    */
30  Student searchByLastname(String lastname);

32  /**
    * Search a person within the PhoneBook by number
34  *
    * @param phone The phone to be searched
36  * @return The person found, null otherwise
    */
38  Student searchByNumber(String phone);

40  /**
    * Delete a person from the PhoneBook
42  *
    * @param phone The phone number to be searched.
44  */
    void deleteByNumber(String phone);
46 }
```



```
package com.oop.collections.phonebook;

2
import java.util.ArrayList;

4
/**
6  * A PhoneBook implementation internally using ArrayList. Slow!
    *
8  */
public class PhoneBookList implements PhoneBook {
10     ArrayList<Student> phoneBook;

12     public PhoneBookList() {
        phoneBook = new ArrayList<>();
14     }

16     @Override
    public void addPerson(Student p) {
18         /* TODO */
    }
```



```
20 }
21
22 @Override
23 public Student searchByName(String name) {
24     /* TODO */
25 }
26
27 @Override
28 public Student searchByLastname(String lastname) {
29     /* TODO */
30 }
31
32 @Override
33 public Student searchByNumber(String phone) {
34     /* TODO */
35 }
36
37 @Override
38 public void deleteByNumber(String phone) {
39     /* TODO */
40 }
```



```
package com.oop.collections.phonebook;
2
import java.util.HashMap;
4
/**
6  * A PhoneBook implementation internally using HashMap
7  */
8 public class PhoneBookMap implements PhoneBook {
9     HashMap<String, Student> phoneBook;
10
11     public PhoneBookMap() {
12         /* TODO */
13     }
14
15     // We use the phone number as key because it is unique
16     public void addPerson(Student s) {
17         /* TODO */
18     }
19
20     public Student searchByName(String name) {
21         /* TODO */
22     }
23
24     public Student searchByLastname(String lastname) {
```



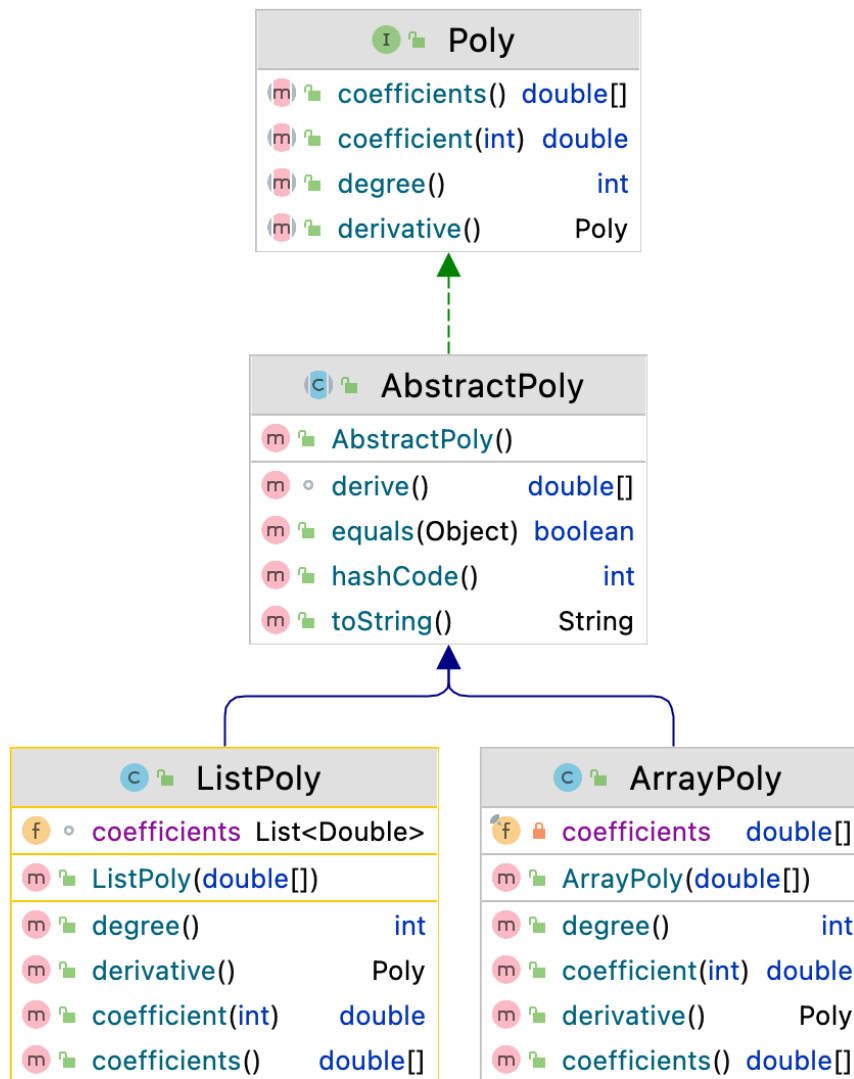
```
26 }  
  
28 public Student searchByNumber(String phone) {  
    /* TODO */  
30 }  
  
32 public void deleteByNumber(String phone) {  
    /* TODO */  
34 }  
}
```



```
1 package com.oop.collections.phonebook;  
  
3 /**  
 * The PhoneBook Interface defines the functionalities of a  
5 * basic phone book.  
 *  
7 * Provide two different implementations of the PhoneBook interface  
 * working with the use case below. The first, (a) internally uses  
9 * an ArrayList, the second (b) internally uses an HashMap.  
 */  
11 public class TestApp {  
    public static void main(String[] args) {  
13        PhoneBook pb = new PhoneBookMap();  
  
15        // Comment this line for switching implementation  
        // PhoneBook pb = new PhoneBookArray();  
17        pb.addPerson(new Student("Nicola", "Bicocchi", "34567"));  
        pb.addPerson(new Student("Marco", "Rizzo", "45243"));  
19        pb.addPerson(new Student("Luisa", "Poppi", "24564"));  
  
21        System.out.println(pb.searchByName("Marco"));  
        System.out.println(pb.searchByLastname("Poppi"));  
23  
        // Do not exist!  
25        System.out.println(pb.searchByNumber("1111"));  
  
27        // Delete an element!  
        pb.deleteByNumber("24564");  
29        System.out.println(pb.searchByLastname("Poppi"));  
    }  
31 }
```


2.3 Polynomials

Write code for an application designed as shown in the following class diagram.



```

1 package com.oop.collections.polynomials;
2
3 /**
4  * Interface representing a polynomial with arbitrary grade
5  */
6 public interface Poly {
7
8     /**
9      * Returns the degree of the polynomial
10     */
11 }
  
```



```

11  *
12  * @return the degree of the polynomial
13  */
14  int degree();

15  /**
16  * Returns a new polynomial which is the derivative of the current
17  * object. The call invoked on object (1), returns object (2)
18  * (1)  $c_0 + c_1 * x + \dots + c_n * x^n$ 
19  * (2)  $c_1 + 2c_2 * x + \dots + nc_n * x^{(n-1)}$ 
20  *
21  * @return Returns a new polynomial which is the derivative of the
22  * current object
23  */
24  Poly derivative();

25  /**
26  * Returns the coefficient of the monomial with the specified degree
27  *
28  * @param degree The degree to be queried (get the coefficient)
29  * @return The coefficient of the monomial with the specified degree
30  */
31  double coefficient(int degree);

32  /**
33  * Returns a double[] containing all the coefficients
34  *
35  * @return A double[] containing all the coefficients
36  */
37  double[] coefficients();
38  }

```



```

1  package com.oop.collections.polynomials;
2
3  import java.util.Objects;
4
5  /**
6  * An abstract class providing an implementation for shared parts of
7  *   ↪ ArrayPoly
8  *   and ListPoly
9  */
10 public abstract class AbstractPoly implements Poly {
11
12     double[] derive() {
13         /* TODO */
14     }
15 }

```



```
16  @Override
    public boolean equals(Object o) {
    18      /* TODO */
    }

    20  @Override
    public int hashCode() {
    22      return Objects.hashCode(coefficients());
    }

    24  @Override
    26  public String toString() {
        /* TODO */
    28  }
}
```



```
1  package com.oop.collections.polynomials;

3  /**
   * Class representing a polynomial with coefficients stored as on
   5  * array of doubles
   */
7  public class ArrayPoly extends AbstractPoly {
    private final double[] coefficients;

    9

    public ArrayPoly(double[] coefficients) {
    11      /* TODO */
    }

    13

    15  @Override
    public int degree() {
        /* TODO */
    17  }

    19  @Override
    public Poly derivative() {
    21      /* TODO */
    }

    23

    25  @Override
    public double coefficient(int degree) {
        /* TODO */
    27  }

    29  @Override
    public double[] coefficients() {
    31      /* TODO */
    }
```



```
33 }
```



```
1 package com.oop.collections.polynomials;

3 import java.util.ArrayList;
  import java.util.List;

5
6 /**
7  * Class representing a polynomial with coefficients stored as a list
8  */
9 public class ListPoly extends AbstractPoly {
10     List<Double> coefficients;

11
12     public ListPoly(double[] coeffs) {
13         /* TODO */
14     }

15
16     @Override
17     public int degree() {
18         /* TODO */
19     }

20
21     @Override
22     public Poly derivative() {
23         /* TODO */
24     }

25
26     @Override
27     public double coefficient(int degree) {
28         /* TODO */
29     }

30
31     @Override
32     public double[] coefficients() {
33         /* TODO */
34     }
35 }
```



```
1 package oop.collections.exercises.polynomials;

3 /**
4  * Develop two classes , namely ArrayPoly and ListPoly ,
```



```

5  *   for managing polynomials.
   *   More specifically, the two classes must exhibit the same
7  *   functionalities (they both implement the Poly Interface) but
   *   using different mechanisms internally.
9  *   <p>
   *   Given a generic polynomial,  $c_0 + c_1 * x + \dots + c_n * x^n$ 
11 *   <p>
   *   ArrayPoly stores a polynomial using double[] (c0..c_n).
13 *   <p>
   *   ListPoly, stores a polynomial using List<Double> (c0..c_n).
15 *   <p>
   *   Both classes must work with the main method provided below.
17 */
public class TestApp {
19     public static void main(String[] args) {

21         Poly ap = new ArrayPoly(new double[]{1, 3, 4, 8});
         Poly lp = new ListPoly(new double[]{1, 3, 4, 8});

23
         System.out.println("ap = " + ap);
25         System.out.println("lp = " + lp);

27         if (ap.equals(lp) && lp.equals(ap))
         System.out.println("ap == lp");
29         else
         System.out.println("ap != lp");

31
         ap = ap.derivative();
33         System.out.println("ap' = " + ap.toString());

35         ap = ap.derivative();
         System.out.println("ap'' = " + ap.toString());

37
         lp = lp.derivative();
39         System.out.println("lp' = " + lp.toString());

41         lp = lp.derivative();
         System.out.println("lp'' = " + lp.toString());
43     }
}

```